Chapter 4

OPTWIN: Drift Identification with Optimal Sub-Windows

4.1 Contribution Statement

This chapter corresponds to the extended version of the paper "OPTWIN: Drift Identification with Optimal Sub-Windows" (Tosi and Theobald, 2024) authored by Mauro Dalle Lucca Tosi and Martin Theobald - © 2024 IEEE. Below, we provide the CRediT (Contributor Roles Taxonomy) [1] statements correspondent to each author contribution.

- **Mauro Dalle Lucca Tosi:**
  - Conceptualization;
  - Methodology;
  - Software;
  - Validation;
  - Formal analysis;
  - Investigation;
  - Data Curation;
  - Writing - Original Draft;
  - Visualization;
- **Martin Theobald**
  - Formal analysis;
  - Resources;
  - Writing - Review & Editing;
  - Supervision;
  - Project administration;
  - Funding acquisition

4.2 Chapter Contextualization

This chapter corresponds to the extended version of the paper "OPTWIN: Drift Identification with Optimal Sub-Windows" (Tosi and Theobald, 2024). It proposes

[1] https://credit.niso.org/

OPTWIN, an error-based drift detector algorithm that detects concept drifts from classification and regression problems while producing a low false positive rate. The development of such drift detectors is essential for optimizing the utilization of computational resources and preventing unnecessary retraining of ANN models that have already converged and have not experienced concept drift.

4.3 Introduction

Online Learning (OL) is an area of research that gained an increasing amount of attention in academia and industry over the past years. OL is a sub-field of Machine Learning (ML) in which an underlying ML technique aims to constantly update its model's parameters from an incoming data stream under limited time and space constraints. Ideally, OL methods are trained in real-time and thereby aim to maximize the prediction accuracy of their models based on bounded windows of data instances they have previously seen and which they may see in the near future (Hoi et al., 2021). Use-cases of OL are varied and include online video segmentation (Koner et al., 2023), spam detection (Fernández Riverola et al., 2007), oral hygiene rewarder (Trella et al., 2023), and many others (Wang et al., 2023; Li et al., 2023; Mandal et al., 2023).

The presence of *concept drifts* is one of the numerous challenges when processing data streams in real-time. Concept drifts are commonly defined as unforeseeable changes in the statistical properties of the incoming data stream over time (Lu et al., 2018). Such a change may have different sources and types, and it may involve different adaptation strategies (cf. Section 4.4.1). In practice, concept drift is the key phenomenon that impairs the performance of pre-trained (and hence static) ML models over data streams.

The most common form of drift detection is the *error rate-based* one. This type of drift detection uses the difference between the number or magnitude of historical prediction errors and new prediction errors to determine if a concept drift occurred (Lu et al., 2018). The most popular error rate-based algorithms are ADWIN (Bifet and Gavalda, 2007), DDM (Gama et al., 2004), and their variations as (Baena-Garcıa et al., 2006; Barros et al., 2017). Those and newer drift detectors (Wu et al., 2021; Hidalgo, Mariño, and Barros, 2019) are known for being lightweight and identifying concept drifts with an outstanding true-positive rate. However, those algorithms have a high false-positive rate. Additionally, most of them work with classification problems only, with few being suited for regression problems. Therefore, as concluded by Bayram et al. (Bayram, Ahmed, and Kassler, 2022), the development of drift detectors which return a low false-positive rate and are suited for both classification and regression are open research problems in this area.

As opposed to previous approaches, which track concept drifts based on the *means* of past prediction errors, we argue that also changes in the *standard deviations* of those errors should trigger concept drifts. Take, as a simple example, a regressor that outputs the following errors at window W_0:

$$W_0 = \langle 0.3; 0.7; 0.7; 0.3; 0.3; 0.7; 0.5; 0.5 \rangle$$

While, at window W_1, it outputs the following errors:

$$W_1 = \langle 0.0; 1.0; 1.0; 0.0; 1.0; 0.0; 0.0; 1.0 \rangle$$

Current drift detectors like ADWIN would not consider this as a concept drift, as the error means μ_{W_0}, μ_{W_1} over both windows are equal to 0.5, whereas we—intuitively—understand that something changed in the error stream and, therefore, a concept drift should be flagged.

In this paper, we propose the "OPTimal WINdow" drift detector (OPTWIN). It is a sliding-window algorithm that analyzes the error rates (either binary or real-valued) produced by an underlying ML algorithm. It calculates the *optimal cut* of a sliding window W to divide it into two sub-windows W_{hist} and W_{new}. It then performs the well-known t- (for means) and f-tests (for standard deviations) to determine whether the two sub-windows exhibit a statistically significant difference in either their means or standard deviations, respectively. OPTWIN has the following two main features: (1) it can calculate the *optimal cut* based only on the length of W in $\mathcal{O}(1)$; (2) it uses both the errors' means and standard deviations to flag drifts. Furthermore, we provide detailed and rigorous guarantees of OPTWIN's performance in terms of its true-positive (TP), false-positive (FP), and false-negative (FN) rates, as well as in its drift-identification delay.

To assess OPTWIN, we used the popular MOA framework (Bifet et al., 2010) and compared OPTWIN to the most studied drift-detection frameworks in the literature: ADWIN (Bifet and Gavalda, 2007), DDM (Gama et al., 2004), EDDM (Baena-Garcıa et al., 2006), STEPD (Nishida and Yamauchi, 2007) and ECDD (Ross et al., 2012). Using MOA, we compared the *precision*, *recall* and *F1-score*, as well as the *delay* of all drift detectors over both sudden and gradual drifts. Furthermore, we also trained a Naïve Bayes (NB) classifier on MOA that adapts itself based on the drift detectors and compared its results on various synthetic datasets (STAGGER (Schlimmer and Granger, 1986), RANDOM RBF (Bifet et al., 2009), and AGRAWAL (Agrawal, Imielinski, and Swami, 1993)) and real-world ones (Covertype and Electricity) (Blackard and Dean, 1999; Harries, 1999). Additionally, we likewise assessed OPTWIN on a Neural Network (NN) use-case. Specifically, we pre-trained a Convolutional NN (CNN) with the CIFAR-10 (Krizhevsky, Hinton, et al., 2009) image dataset and simulated an end-to-end OL scenario with concept drifts by using OPTWIN and ADWIN as drift detectors.

Our results show that OPTWIN reliably identifies both sudden and gradual drifts in classification and regression problems. Furthermore, OPTWIN is the drift detector with the best F1-score when compared to the baselines. This is due to its higher precision, which indicates a low FP rate. With respect to the drift-identification delay, there was no drift detector that was superior to the others in a majority of the datasets. In terms of run-time, OPTWIN in combination with the CNN training pipeline is 21% faster than a similar ADWIN pipeline due to OPTWIN's lower FP rate, which leads to a significantly reduced amount of re-training iterations. Therefore, our experiments indicate that OPTWIN identifies concept drifts with a similar delay as other drift detectors but maintains a higher precision and recall in the drift detection, which reduces the overall run-time of OL pipelines that trigger a re-training of their models upon each detected concept drift.

4.4 Background & Related Works

We next formally introduce the problem of concept drift and review its principal characteristics (cf. Section 4.4.1). We also discuss the most important related approaches for concept-drift detection from the literature, which serve as inspiration and baselines for our work (cf. Section 4.4.2).

4.4.1 Background

A concept drift may exhibit different characteristics, whose understanding is essential for a fast and reliable drift detection. Consider an unbounded data stream that receives *features* $z_i \in Z$ and *labels* $y_i \in Y$ in the form of pairs of instances (z_i, y_i) used for training an underlying ML model. At *time* t, these instances follow a certain distribution $P_t(Z, Y)$. A concept drift is the change of this distribution over time. Formally, a concept drift occurs at time $t+1$ iff $P_t(Z, Y) \neq P_{t+1}(Z, Y)$ (Lu et al., 2016; Lu et al., 2018; Gama et al., 2014). The underlying data distribution $P_t(Z, Y)$ may also be decomposed and expressed as a product of probabilities $P_t(Z, Y) = P_t(Z) \times P_t(Y|Z)$ via the well-known *chain rule of probability*. Thus, a concept drift may come from two *sources*: (1) $P_t(Z) \neq P_{t+1}(Z)$, which is known as a *virtual drift* because it does not impact the decision boundaries of the learner; and (2) $P_t(Y|Z) \neq P_{t+1}(Y|Z)$, which is known as an *actual drift* because it directly impacts the learner's accuracy (Lu et al., 2016; Lu et al., 2018; Gama et al., 2014). It is also possible to have both drift sources simultaneously (Lu et al., 2018).

Another important characteristic of a concept drift is its *type*. Concept drifts can be (1) *sudden*; (2) *incremental*; (3) *gradual*; and (4) *reoccurring* (see Figure 4.1) (Lu et al., 2018; Gama et al., 2014). Sudden drifts occur when the probability distribution changes completely within a single step. Incremental drifts occur when the distribution $P_t(Z, Y)$ changes incrementally until its convergence. Gradual drifts occur when the new distribution gradually replaces the old one. Reoccurring drifts occur when distributions can reoccur after some time.

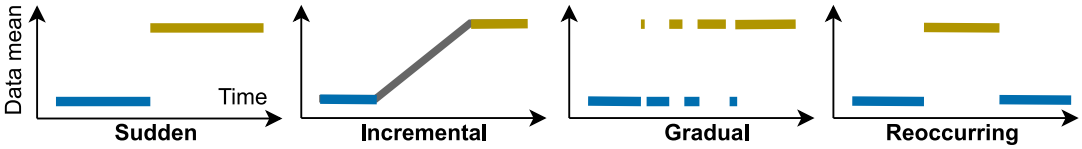


FIGURE 4.1: Concept drift types.

To recover from concept drifts, one may adapt the learner according to different strategies. The selection of the appropriate strategy depends not only on the type and source of the drifts but also on the use-case and the adopted learner. A common strategy is to train a new model with the latest data points whenever a drift is identified. Another strategy is to adjust the current learner instead of training a new model from scratch. Moreover, it is also possible to have an ensemble of learners, which is primarily useful for adapting to reoccurring drifts (Lu et al., 2018).

The most popular procedure to evaluate learning algorithms that handle concept drifts is the *prequential procedure*. Prequential is an evaluation scheme in which each data point is used for testing before training the learning algorithm. Therefore, it is not necessary to know when a drift occurred to perform the evaluation, which is useful when using real-world datasets that do not have labeled drifts (Lu et al., 2018). Regarding the metrics used to evaluate the drift detector, one may refer to the following ones: TPs, FPs and FNs (and hence *precision*, *recall* and *F1-score*) in the detection rates, as well as the *delay* of the detection (Lu et al., 2018; Dasu et al., 2006).

4.4.2 Related Works

ADWIN (Bifet and Gavalda, 2007) (for "ADaptive WINdowing") is an algorithm with a sliding window W that takes as input a confidence level $\delta \in (0, 1)$ and an unbounded sequence of real values $X = \langle r_1, r_2, ..., r_i, ... \rangle$ from a data stream. Each input r_i is expected to be in $[0, 1]$ and is stored in a finite window $W \subset X$. The idea of

the algorithm is that a concept drift occurs whenever there are two subsequent sub-windows of W with an average difference in their mean above ϵ_{cut}, which is guaranteed to yield a *confidence level* of δ. If a drift is detected, a part of W is dropped, and the algorithm is ready to resume the drift detection. ADWIN has guaranteed bounds on all error distributions. However, the observed errors usually tend to follow a normal distribution. Thus, they tightened their ϵ_{cut} accordingly and evaluated the algorithm taking those new bounds. Furthermore, as ADWIN checks multiple sub-windows of W, its complexity is $\mathcal{O}(\log |W|)$ per iteration (where an iteration is ideally triggered for each new element that arrives from the stream).

DDM (Gama et al., 2004) (for "Drift Detection Method") also receives as parameter a value δ related to the confidence level of the algorithm, and a sequence of data instances X which are available one by one from a data stream. However, the data stream is expected to follow a binomial distribution, as its values represent the number of errors in a sample of n instances. Therefore, it is primarily suited for classification problems rather than for regression. The main idea of DDM is that, as the number of samples increases, the number of errors from a learner will decrease. Therefore, if the error rate from the learner increases above a certain threshold, it means that a concept drift occurred. Thus, DDM tracks the minimal error rate p_{min} from X and its respective standard deviation s_{min}. Then, DDM flags a drift at step $i \geq 30$ iff $p_i + s_i \geq p_{min} + \delta s_{min}$. When a drift is detected, the algorithm is reset. One advantage of DDM is its lightweight nature due to storing only p_{min} and s_{min}. However, DDM is known for its decreased performance when detecting slow gradual changes.

EDDM (Baena-Garcıa et al., 2006) (for "Early Drift Detection Method") is similar to DDM but uses the distance between errors to identify concept drifts. Therefore, EDDM is suited only for binary input data, which does not allow it to be used for regression problems. They assume that, as the number of examples increases, the distance between errors will increase accordingly. Therefore, if this distance decreases more than a threshold, a concept drift is flagged. Thus, EDDM tracks the maximum distance between errors p'_{max} and its respective standard deviation s'_{max}. Then, after analyzing at least 30 errors, EDDM flags a drift at step t if $(p'_t + 2\,s'_t)/(p'_{max} + 2\,s'_{max} < \delta)$, with δ again representing a simple form of confidence level for the detected concept drifts. According to the authors, EDDM improves DDM's performance when identifying gradual concept drifts while maintaining good performance on sudden drifts.

STEPD (Nishida and Yamauchi, 2007) (for using the "Statistical Test of Equal Proportions") is a lightweight drift detector that also considers a sequence of data instances X as input. The idea of STEPD is that the recent accuracy of a learner shall be similar to its overall accuracy, otherwise a concept drift occurred. As the name suggests, STEPD checks the equality-of-proportions hypothesis test (Massey and Miller, 2006) by comparing the most recent values from X (kept in a sliding window W) with the other values in X, thus comparing $X_{0:W}$ and $X_{W:|X|-1}$. It also considers that the data follows a normal distribution, and with a confidence level of δ, it can determine if the null hypothesis is rejected and a concept drift is flagged. In this case, the algorithm is reset. The value of W selected by the authors was 30.

ECDD (Ross et al., 2012) is the acronym for "Exponentially Weighted Moving Average" (EWMA) for concept-drift detection. It is a drift-detection method based on EWMA that checks the misclassification rate of its underlying learners. ECDD receives a sequence of data instances X as input, which it assumes to follow a Bernoulli distribution. ECDD is suited only for classification problems, as it only accepts binary data from the data stream. Other than X, ECDD takes as inputs ARL_0, which

is the time one expects between detecting false positives; and λ, which dictates how much weight new data has compared to the old one. Despite being computationally expensive to calculate, ARL_0 can be approximated before starting to process the data stream. ECDD then uses pre-calculated polynomials, ARL_0 and λ to update its estimators. With its updated estimators, ECDD detects if a concept drift occurred or not. Finally, if a drift is detected, the algorithm is reset.

We note that all algorithms described above additionally have a form of warning detection which can be used to assist in the concept drift adaptation. The warning is usually a simple relaxation of the drift threshold which can be used to start training a new learner before the actual concept drift is flagged.

4.5 OPTWIN – Optimal Window Concept Drift Detector

OPTWIN is the new concept-drift detector which we propose in this paper. It is an error rate-based drift detector, which tracks the error rates produced by an OL learner in a sliding window W. Its name stands for "OPTimal WINdow" due to its calculation of the optimal split of the sliding window. We consider this split "optimal" because it is detected for the first element in W at which a statistically significant difference between either the means or the variances of the error rates (based on the common t- and the f-tests) among two sub-windows of W occurs. Specifically, W keeps growing until either a concept drift is detected or a maximum user-defined size w_{max} is reached. Based on the sliding-window size $|W|$ and a predefined confidence level δ, OPTWIN calculates the optimal point to divide W into "historical" (W_{hist}) and "new" (W_{new}) data points.

4.5.1 Setup & Parameters

Without loss of generality, we consider a typical OL scenario, in which OPTWIN receives as input a sequence of real numbers $r_1, r_2, ..., r_i, ...$ from a possibly unbounded data-stream X. Thus, the algorithm is assumed to access only one element r_i at time i from the stream, and it buffers previously seen elements in a *sliding window* $W \subset X$ of consecutive events. Moreover, OPTWIN requires as parameters (1) δ, the *confidence level* for the concept-drift detection; (2) w_{max}, the *maximum size* of the sliding window W; and (3) ρ, a parameter we refer to as *robustness*, which we define as the minimum ratio by which $\mu_{W_{new}}$ has to vary in relation to $\sigma_{W_{hist}}$ in order for OPTWIN to consider this variation as a concept drift.

4.5.1.1 Assumptions

- There is no concept drift within the first w_{min} data instances from X (which is needed to initialize OPTWIN).
- A concept drift occurs when the means or the standard deviations within two sub-windows of W are statistically different.
- Within any two sub-windows of W, the values produced by the test statistic of the unequal-variance t-test (Ruxton, 2006) (henceforth called "t_value") follow a t-distribution.
- Within any two sub-windows of W, the values produced by the test statistic of the f-test (Mahbobi and Tiemann, 2016) (henceforth called "f_value") follow an f-distribution.

Our first assumption mitigates the negative impact of outliers when only a few data points are available. The second assumption indicates when we expect OPTWIN to identify concept drifts. The third and the fourth assumptions are generally required to guarantee the validity of the t- and f-tests, respectively, but do not impose limitations on the values ingested from the data stream in practice.

4.5.2 Algorithm

Algorithm 2 OPTWIN

Input parameters:
- δ – confidence level
- ρ – robustness
- w_{max}– max window size

Global variables:
$W = \langle\,\rangle$ – sliding window
$w_{min} = 30$ – min window size
$\beta = 1e^{-5}$ – avoids division by 0

procedure ADDELEMENT(r_i)
  $W \leftarrow W \cup r_i$
  if $|W| < w_{min}$ **then**
    return False
  else if $|W| \geq max_lenght$ **then**
    $W \leftarrow W - W_0$
  $\nu \leftarrow$ OPTIMALCUT($|W|, \rho, \delta^{\frac{1}{4}}$) cf. Equation (4.1)
  $\nu_{split} \leftarrow \lfloor \nu\,|W| \rfloor$
  $W_{hist} \leftarrow W_{0:\nu_{split}}$
  $W_{new} \leftarrow W_{\nu_{split}:|W|-1}$
  // *f-test*
  if $\frac{(\sigma_{W_{new}}+\beta)^2}{(\sigma_{W_{hist}}+\beta)^2} > f_ppf(\delta^{\frac{1}{4}}, \nu|W|-1, (1-\nu)|W|-1)$ **then**
    reset()
    return True
  // *t-test*
  else if $t_value(W_{hist}, W_{new}) > t_ppf(\delta^{\frac{1}{4}}, df)$ **then**
    reset()
    return True

As seen in Algorithm 2, OPTWIN first needs to be initialized by creating an empty sliding window W. Analogously to other drift detectors, it collects w_{min} many first elements from the stream (usually within 30 to 50) to guarantee that it has a minimum amount of data to output statistically relevant drift detections. It also defines a small constant $\beta = 1e^{-5}$, which avoids a division by 0 when added to the standard deviations during the calculation of the f-test.

Second, the ADDELEMENT procedure is called once to process each data element received from the data stream. The procedure starts by inserting the most recent data element r_i into W and by checking whether W has already reached the minimum amount of elements to detect a possible concept drift. If so, it also checks if W increased above w_{max} and removes the oldest element from W if necessary. Up to this point, OPTWIN performs standard queuing operations to maintain its sliding window bounded between w_{min} and w_{max} elements.

Then, OPTWIN calculates ν, which is the optimal splitting point of W into W_{hist} and W_{new}, by solving Equation 4.1 for the highest value of ν in terms of δ', ρ and $|W|$:

$$\rho = t_ppf(\delta', df)\sqrt{\frac{1}{\nu|W|} + \frac{f_ppf(\delta', \nu\,|W|-1, (1-\nu)\,|W|-1)}{(1-\nu)\,|W|}} \tag{4.1}$$

where ρ is the aforementioned, user-defined robustness parameter. During this calculation, OPTWIN uses t_ppf and f_ppf, which are the Probability Point Functions (PPF) of the t- and the f-distributions, respectively. To calculate t_ppf, we apply a confidence of $\delta' = \delta^{\frac{1}{4}}$, which considers the application of the two tests to calculate ν in Equation 4.1 and the two tests on Lines 11 and 14 in Algorithm 2.

Finally, OPTWIN performs the t- and f-tests (in any order) to compare both sub-windows and determine if their means and standard deviations, respectively, belong to the same distribution. If not, a concept drift is flagged and the algorithm is reset. Otherwise, the ADDELEMENT method is called for the next element from the data stream in an iterative manner.

The usage of the t- and f-tests to identify significant changes in the means and standard deviations among series of data values is well-established. However, the novelty of OPTWIN comes from the combination of both tests to calculate ν, and thereby identify where to optimally divide this series of values to perform both tests. In short, by solving Equation 4.1 in terms of ν, we determine the minimum size of W_{new} (the optimal splitting point of W) that statistically guarantees the identification of any concept drift with a robustness of at least ρ when using the t- and f-tests, thereby achieving lower drift-detection delays than other approaches. To calculate ν, other than the confidence level δ, the only values that shall be inputted by the user are w_{max} and ρ. Regarding ρ, when selecting a small value, one may expect to identify smaller drifts in exchange of a higher drift-detection delay. On the other hand, with higher values of ρ, one can expect a smaller detection delay in exchange of missing smaller drifts. In practice, ρ shall be set based on the magnitude and frequency of drifts expected. However, determining the correct ρ to each use case should not be a difficult task, since different ρ's tend to produce similar results (as seen in Section 4.6). Regarding w_{max}, with a higher value, one may expect smaller drift-detection delays in exchange for more memory usage. In practice, when using $\rho = 0.1$, we did not observe a significant variation in $|W_{new}|$ even when increasing w_{max} to more than 25,000 elements.

One important point to note is that we can only calculate ν as the optimal splitting point if W is large enough. Otherwise, we set ν to the middle of W until it grows to the minimum size needed to solve Equation 4.1. Thus, if it exists, it is defined as the highest root of Equation 4.1. Otherwise, it is set to $\nu = 0.5$.

Below, we present Theorem 4.5.1, our main theoretical result. It gives guarantees in terms of OPTWIN's false positive (FP) and false negative (FN) bounds for identifying concept drifts. Theorem's 4.5.1 proof is available in Section 4.8.1.2.

Theorem 4.5.1.

- ***False Positive Bound.*** *At every step, if μ_W and σ_W^2 remain constant within W, OPTWIN will flag a concept drift at this step with a confidence of at most 1-δ.*
- ***False Negative Bound*** *(for mean drift with large enough W). For any partitioning of W into two sub-windows $W_{hist}\,W_{new}$, with $|W| \geq w_{proof}$ and W_{new} containing the most recent elements, if $\mu_{hist} - \mu_{new} > \rho\,\sigma_{hist}$, then, with confidence δ, OPTWIN flags a concept drift in at most $|W| - \nu_{split}$ steps.*
- ***False Negative Bound*** *(for mean drift with small W). For any partitioning of W into two sub-windows $W_{hist}\,W_{new}$, with $w_{min} \leq |W| < w_{proof}$ and W_{new} containing the*

most recent elements, if $\mu_{hist} - \mu_{new} > \rho_{temp}\,\sigma_{hist}$*, then, with confidence* δ*, OPTWIN flags a concept drift in at most* $\frac{|W|}{2}$ *steps.*

- ***False Negative Bound*** *(for standard-deviation drift with any* W*). For any partitioning of* W *into two sub-windows* W_{hist} W_{new}*, with* $|W| \geq w_{min}$ *and* W_{new} *containing the most recent elements, if* $\frac{\sigma^2_{new}}{\sigma^2_{hist}} > f_ppf(\delta', \nu\,|W| - 1, (1-\nu)\,|W| - 1)$*, then, with confidence* δ*, OPTWIN flags a concept drift in at most* ν_{split} *steps.*

4.5.3 Implementation & Analysis

We implemented OPTWIN via a combination of Java and Python scripts as extensions of the MOA (Bifet et al., 2010) (Java) and the River (Montiel et al., 2021) (Python) libraries (both available on Github[2]). We pre-calculated the values of ν, t_ppf, and f_ppf based on a fixed confidence value of $\delta = 0.99$ and for $w_{max} = 25{,}000$, thus $30 \leq |W| \leq 25{,}000$. All pre-calculated variables were stored in lists indexed by the window sizes $|W|$ from which they were calculated. This is possible because those variables, as seen in Equation 4.1, do not depend on the actual error distribution. Thus, it is not necessary to calculate them in real-time. On the other hand, we need to store the sliding window W plus the other three lists of floating point values of size up to w_{max} in memory, which takes a maximum of $w_{max} * 4 * 4$ bytes. Thus, for $w_{max} = 25{,}000$, OPTWIN would require only around 390 KB of memory.

Furthermore, the full calculation of the means and standard deviations from W_{hist} and W_{new} can be avoided. Instead of calculating them from scratch, one only needs to update them incrementally. Moreover, as W is bounded by w_{max}, we can use a circular array to make insertions at the end of the array, deletions from the beginning of the array, and then look up each value in $\mathcal{O}(1)$ time. Therefore, the ADDELEMENT procedure has an overall computational complexity of $\mathcal{O}(1)$ per element ingested from the data stream (assuming a constant cost for the numerical operations involved in resolving Equation 4.1 to ν). Our analytical approach thus provides great potential runtime gains over the iterative search procedure applied, e.g., by ADWIN which requires $\mathcal{O}(\log |W|)$ computations per iteration.

By default, OPTWIN's Algorithm 2 tracks concept drifts that either increase or decrease the means and standard deviations of the variables tracked. However, in an OL scenario, we usually want to update the learner only when the number of errors increases. Therefore, in our implementation, we check if also $\mu_{new} \geq \mu_{hist}$ along with the statistical tests on Lines 11 and 14 of Algorithm 2. In doing so, we consider that a drift occurred only if μ_{new} is higher than μ_{hist} (i.e., when the learner decreased in performance).

4.6 Experiments & Results

In this section, we report our detailed experiments to asses OPTWIN. We compared OPTWIN to the most commonly used baselines for concept-drift detection: ADWIN, DDM, EDDM, STEPD, and ECDD. We additionally investigated more recent techniques (Hidalgo, Mariño, and Barros, 2019; Wu et al., 2021; Barros et al., 2017) but found them transitively compared to the classical techniques mentioned above. That is because they address specific frailties of the classical drift detectors like recurrent drifts and decreased sensitivity when concepts are long, often without statistical improvements over the classical techniques.

[2] https://github.com/maurodlt/optwin

We performed most of the experiments using the common MOA (Bifet et al., 2010) framework which is a Java-based data stream simulator. In addition, we performed experiments on Python to asses OPTWIN on a Neural Network (NN) use case, which would not be possible via MOA.

4.6.1 MOA Experiments

We compared 3 configurations of OPTWIN with the default configurations of our baselines. All OPTWIN configurations had $\delta = 0.99$, $w_{max} = 25{,}000$, and precomputed values for ν, t_ppf, and f_ppf (as described on Section 4.5.3). The difference among the OPTWIN configurations is only on ρ, which we varied between 0.1, 0.5, and 1.0 to better understand how our robustness parameter affects OPTWIN in practice.

We performed two types of experiments on MOA. The first one uses the "Concept Drift" interface, in which MOA creates a stream of data points (either binary or non-binary) and produces a concept drift that can be sudden or gradual. We later refer to those experiments according to their data input and their drift type.

The second type of experiment uses the "Classification" interface. It generates data streams based on both synthetic datasets (STAGGER (Schlimmer and Granger, 1986), RANDOM RBF (Bifet et al., 2009), and AGRAWL (Agrawal, Imielinski, and Swami, 1993)) and real-world ones (Electricity (Blackard and Dean, 1999) and Covertype (Harries, 1999)). The idea of these experiments is to train a classifier that is reset every time a concept drift is detected by a drift detector. We chose MOA's built-in Naïve Bayes (NB) classifier for its simplicity, which facilitates the analysis of the drift-detection results. For the synthetic datasets, we generate data streams with 100,000 data points with drifts occurring every 20,000 data points (either sudden or gradual). For the real-world data sets, the drifts are already present and have an unknown location on the stream.

Tables 4.1 and 4.2 present the comparison among drift detectors on the "Concept Drift" and the "Classification" interfaces respectively. We repeated each experiment 30 times and compared their average TP, FP, and FN rates to compute their micro-average precision, recall, and F1-score, along with their average drift-detection delay (in terms of the number of streamed elements between the occurrence of a known concept drift and its identification by the detector). Note that we did not include in Table 4.2 real-world datasets because it is not possible to calculate the above-mentioned metrics without knowing the drift's locations, which are only given on the synthetic datasets. Moreover, when using the "Classification" interface, we observed a divergence between the starting and ending points of gradual concept drifts in the MOA documentation and in practice. Therefore, we did not include those configurations in Table 4.2.

Table 4.3 shows the average metrics obtained by each drift detector on Tables 4.1 and 4.2. Based on the results aggregated in Table 4.3, we can observe a high F1-score for OPTWIN when compared to the baselines. In fact, with $\rho \leq 0.5$, OPTWIN's average F1-score is over 95%, followed by OPTWIN with $\rho = 1.0$ achieving 89.4%, DDM with 85.6%, ADWIN reaching 67% and the other detectors less than 50%. We assert this due to OPTWIN's low FP rate which produces a high precision and F1-score, respectively. We further compared the F1-scores of all configurations of OPTWIN with the two drift detectors that can be used for regression problems (ADWIN and STEPD), which showed OPTWIN to be superior based on a one-tailed Wilcoxon signed-rank test with $\alpha = 0.05$ in a statistically significant manner.

		gradual drift			sudden drift		
Setting	**Detector**	**Delay**	**FP**	**F1**	**Delay**	**FP**	**F1**
	ADWIN	280	16.33	60%	46.33	1.87	52%
	DDM	365	0.37	93%	64.50	0.20	91%
	EDDM	148	6.33	71%	26.77	6.57	23%
binary	STEPD	180	8.00	37%	**0.40**	6.67	83%
data	ECDD	**117**	5.37	42%	5.47	2.67	42%
	OPTWIN$_{\rho=0.1}$	328	0.30	94%	75.13	**0.00**	**100**%
	OPTWIN$_{\rho=0.5}$	278	0.10	98%	28.17	**0.00**	**100**%
	OPTWIN$_{\rho=1.0}$	317	**0.07**	**99**%	18.33	0.33	86%
	ADWIN	145	**0**	**100**%	25	2	50%
	DDM	-	-	-	-	-	-
non-	EDDM	-	-	-	-	-	-
binary	STEPD	41	150	18%	21	32	6%
data	ECDD	-	-	-	-	-	-
	OPTWIN$_{\rho=0.1}$	**2**	**0**	**100**%	**1**	**0**	**100**%
	OPTWIN$_{\rho=0.5}$	**2**	**0**	**100**%	**1**	**0**	**100**%
	OPTWIN$_{\rho=1.0}$	**2**	**0**	**100**%	**1**	**0**	**100**%

TABLE 4.1: Statistics of drift identification in classification experiments on synthetic datasets.

Setting	**Detector**	**Delay**	**FP**	**F1**
	ADWIN	31.00	0.30	96%
	DDM	6.33	0.17	98%
sudden	EDDM	40.01	**0.00**	**100**%
STAGGER	STEPD	17.39	31.40	20%
	ECDD	0.72	0.47	94%
	OPTWIN$_{\rho=0.1}$	0.76	0.27	97%
	OPTWIN$_{\rho=0.5}$	**0.72**	0.43	95%
	OPTWIN$_{\rho=1.0}$	**0.72**	0.60	93%
	ADWIN	169	6	46%
	DDM	536	2	67%
sudden	EDDM	**53**	11	33%
RANDOM	STEPD	281	25	24%
RBF	ECDD	71	174	4%
	OPTWIN$_{\rho=0.1}$	315	**0**	**86**%
	OPTWIN$_{\rho=0.5}$	187	**0**	**86**%
	OPTWIN$_{\rho=1.0}$	1,931	1	57%
	ADWIN	**229**	4.23	65%
	DDM	1,875	0.90	79%
sudden	EDDM	5,370	17.20	20%
AGRAWAL	STEPD	838	23.47	25%
	ECDD	465	153.6	5%
	OPTWIN$_{\rho=0.1}$	371	0.63	**93**%
	OPTWIN$_{\rho=0.5}$	350	0.77	88%
	OPTWIN$_{\rho=1.0}$	630	**0.27**	91%

TABLE 4.2: Statistics of drift identification in regression experiments on synthetic datasets.

Detector	Average Delay	FP	F1
ADWIN	132.19	4.39	67%
DDM	569.37	0.73	86%
EDDM	1127.56	8.22	49%
STEPD	196.97	34.93	30%
ECDD	131.84	67.22	37%
OPTWIN$_{\rho=0.1}$	156.13	**0.17**	**95**%
OPTWIN$_{\rho=0.5}$	**120.98**	0.19	**95**%
OPTWIN$_{\rho=1.0}$	414.29	0.32	89%

TABLE 4.3: Average statistics of drift identification on synthetic datasets.

Regarding the drift-detection delay, OPTWIN with $\rho = 0.5$ is the drift detector with the smallest delay, taking on average 121 iterations to detect a concept drift; followed by ECDD, ADWIN, and OPTWIN at $\rho = 0.1$ taking 131, 132 and 156 iterations each; the other detectors took on average between 197 and 1,128 iterations to detect a drift, with OPTWIN at $\rho = 1.0$ taking 414 on average. Observe that, the higher the FP rate of a drift detector, the higher are also its chances of identifying a drift earlier. Therefore, when analyzing ECDD's average drift delay of 131 iterations, we have to consider that it had an average of 67 FPs per run, in contrast to an average of less than 0.4 for any of the OPTWIN configurations. Furthermore, considering that DDM, EDDM, and ECDD depend on binary data, they were not included in the experiments using "non-binary" datasets.

We can better visualize the difference between the drift detectors in Figure 4.2, which represents one of the 30 runs of the "sudden binary drift" configuration (we selected the run with results closest to the ones on the observed average). In Figure 4.2, we can see the relatively high FP rate of the EDDM, ADWIN and ECDD detectors when compared to OPTWIN, DDM and STEPD. Moreover, we can see that OPTWIN's detection delay decreases as ρ increases.

In Figure 4.3, we can once again see OPTWIN and DDM identifying all the drifts with a low FP rate. However, in this scenario, ADWIN also produces only a few FPs, which we noticed to occur immediately after correctly identifying a concept drift. This is due to the time it takes to adapt its window size and remove data points from before the concept drift. In this use case, ECDD and STEPD produce almost "random guesses" on the location of the concept drifts.

Additionally, we provide illustrations of the remaining results described in Table 4.4 in Section 4.8.2.

4.6.2 Classification Experiments

In the "Classification" experiments (still using MOA as platform), we can analyze the average accuracy achieved by the NB classifier when varying the drift detectors (cf. Table 4.4). The idea is that the better a drift detector's performance, the better the classifier can adapt to the concept drifts, thus generating a higher prediction accuracy. However, the fast and accurate detection of the drifts did not always traverse to a better accuracy for the classifier. We can observe this by comparing Tables 4.3 and 4.4, in which experiments using drift detectors that produced a low F1-score in Table 4.3 achieved a good accuracy in Table 4.4. Moreover, the drift detectors with the best accuracy on real-world data sets were the ones that detected more

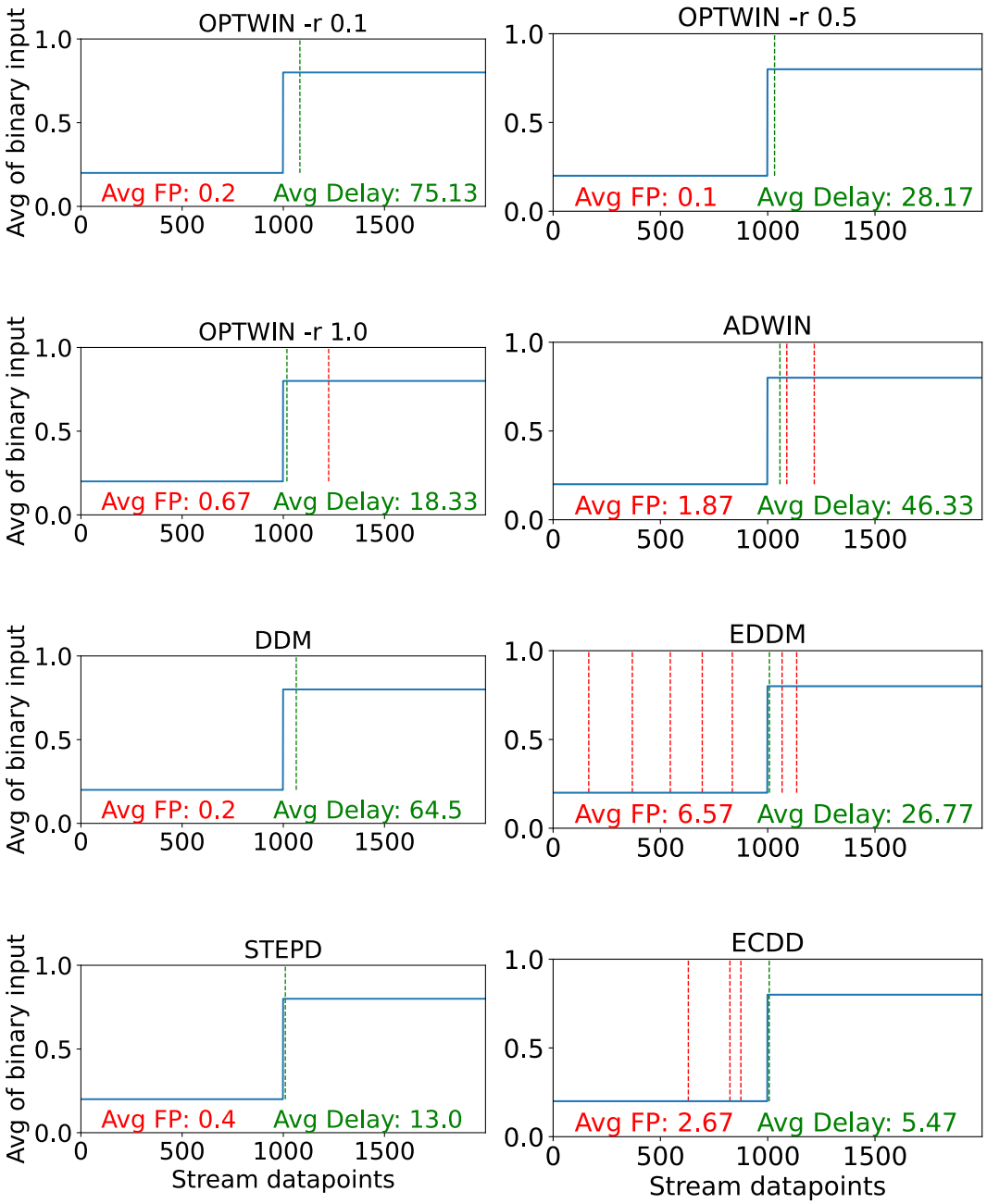


FIGURE 4.2: Sudden binary drift detection with average FP rates compared to drift-detection delays.

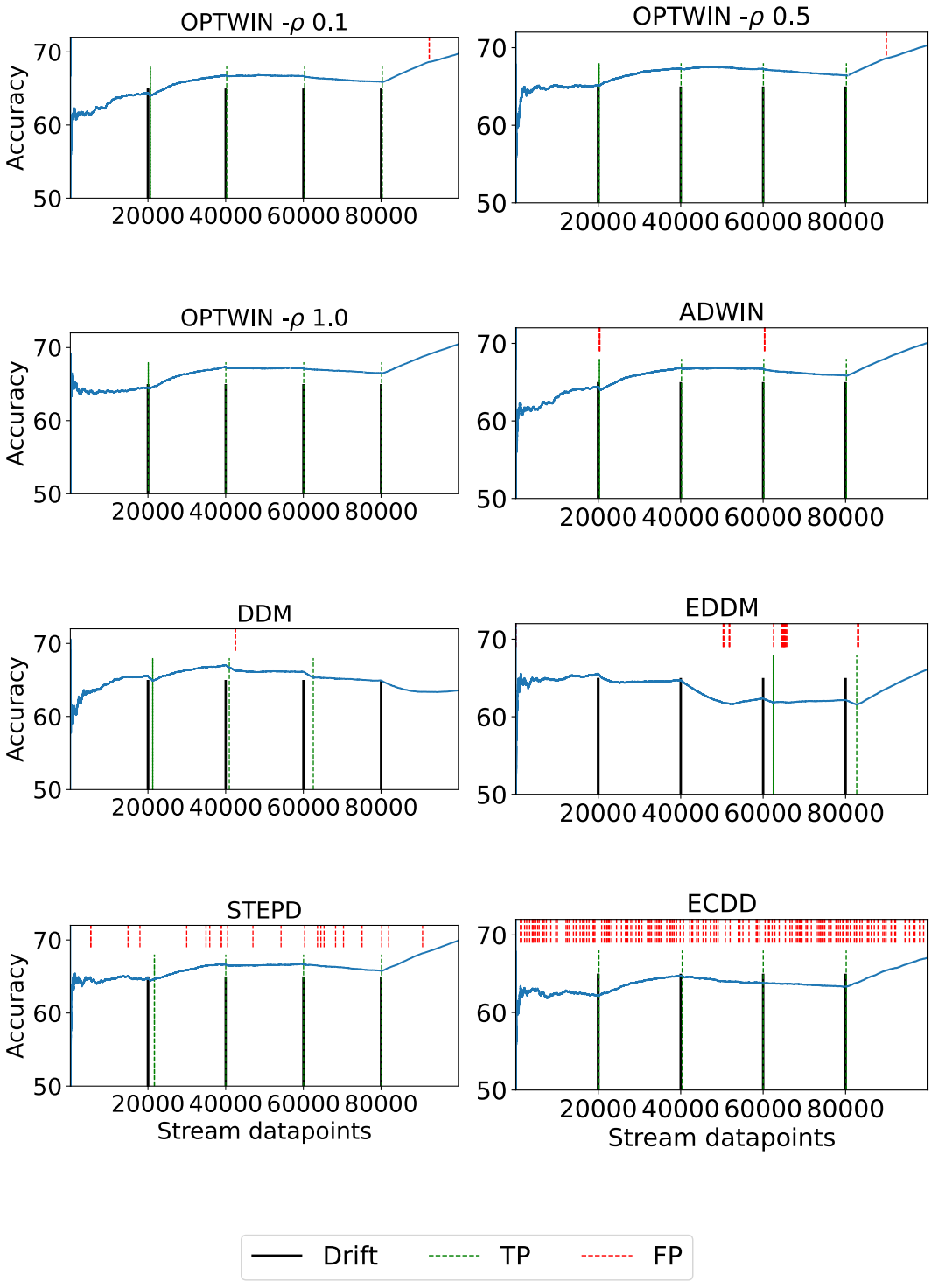


FIGURE 4.3: TP and FP rates for drift detection on the AGRAWAL dataset with sudden concept drifts.

drifts, which is why ECDD achieved such good accuracy (being the detector with the higher amount of FPs). For example, ECDD detected 426 drifts on the Electricity dataset—over twice the amount of other algorithms.

Drift	STAGGER		Random RBF		AGRAWL		Elec	Cove
Detector	Sud	Grad	Sud	Grad	Sed	Grad		
None	66.31	66.31	58.69	58.69	57.94	57.93	73.36	60.52
ADWIN	99.89	98.78	69.74	69.48	**70.22**	69.89	80.01	82.49
DDM	**99.96**	98.67	68.18	68.19	65.53	65.80	81.18	88.03
EDDM	99.87	**98.83**	65.99	66.92	65.46	65.31	84.83	86.08
STEPD	98.74	98.78	**70.92**	**70.49**	69.48	69.13	84.49	87.53
ECDD	**99.96**	98.71	68.64	68.52	67.03	66.80	**86.76**	**90.16**
OPTWIN$_{\rho=0.1}$	**99.96**	98.72	69.65	69.54	70.11	**69.94**	79.99	83.29
OPTWIN$_{\rho=0.5}$	**99.96**	98.51	69.77	69.48	69.91	69.27	83.30	85.63
OPTWIN$_{\rho=1.0}$	**99.96**	98.36	69.67	68.48	69.44	68.61	82.96	86.31

TABLE 4.4: Accuracy of NB on synthetic and real-world datasets. "Sud"and "Grad" representing sudden and gradual concept drifts respectively. "Elec" and "Cove" representing the Electricity and the Covertype datasets.

4.6.3 Regression Experiments on Neural Networks

To further explore OPTWIN's behavior in a regression scenario, we compared it with ADWIN for identifying drifts from the loss of a CNN. We chose ADWIN as our baseline because it was the drift detector with the best F1-score and smallest drift-identification delay among the ones that do not require binary inputs (thus excluding DDM, EDDM, and ECDD). To simplify the reader's understanding of the generation of the concept drift, instead of training a regression problem on a CNN, we here focused on image classification by swapping the labels of the images. Thus, we provoked 4 concept drifts by swapping the labels of two classes of images every 20% of the simulated data stream. For example, after 62,480 iterations, we swapped the labels between images from "cats" to "horses". Nevertheless, as the drift detectors track the loss of the CNN, the type of the problem or the network architecture should not affect the experiment.

We pre-trained an image classification model (TensorFlow, 2022a) with the CIFAR-10 (Krizhevsky, Hinton, et al., 2009) data set during 100 epochs, achieving an average of 89% training accuracy over 3 different runs. Then, we simulated an OL scenario with concept drifts. Our data stream was formed by batches of 32 images from the CIFAR-10 dataset; with a total of 312,400 data points (equivalent to 100 epochs). We simulated our 4 concept drifts by swapping the labels of two classes every 62,480 iterations (the equivalent of 20 epochs). At every iteration, the model classified the 32 images and outputted the loss of the batch. We inputted this loss into the drift-detection algorithm. If a drift was detected, the next 9,372 batches of images (the equivalent of 3 epochs) were used for fine-tuning the model (thus adapting it to the concept drift). Therefore, the goal was for the drift detector to identify the 4 concept drifts that we simulated, triggering the fine-tuning of the model for a total of 12 epochs (3 epochs per drift).

In Figure 4.4, we compare OPTWIN and ADWIN under the setting described above. First, we note that ADWIN's high FP rates made it re-train the model for much longer than OPTWIN. In comparison, ADWIN identified 15 concept drifts

(with 11 FPs), thus triggering model fine-tuning for 61,562 iterations which took in total 945 seconds. In contrast, OPTWIN identified just 5 drifts (with 1 FP), thus triggering the model fine-tuning for 23,430 iterations which took 781 seconds. We note that OPTWIN's running time per iteration is superior to ADWIN, $1e^{-5}$ against $6e^{-6}$ seconds. However, OPTWIN still ends up speeding up the OL pipeline whenever re-training is triggered by the concept-drift detection (21% faster in this use case). This is because the training of a learner is usually computationally more expensive than the drift detection. Thus, with fewer FPs, the total training time can be reduced substantially.

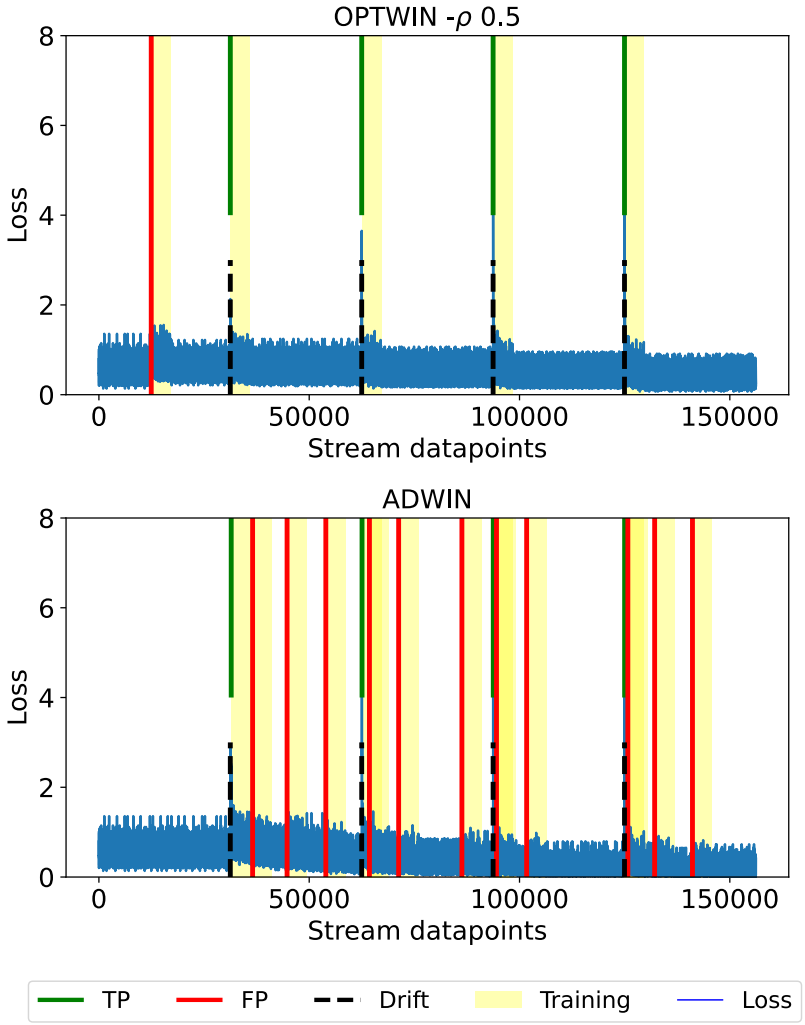


FIGURE 4.4: Sudden drift detection over the loss of a CNN.

4.7 Conclusion

In this paper, we presented OPTWIN, a novel concept-drift detector that uses a sliding window of errors to identify concept drifts in both classification and regression problems with a low FP rate. OPTWIN's novelty relies on the assumption that changes also in the variances of the elements ingested from a data stream can be an indication of concept drift. This assumption lets it optimally divide its sliding window in $\mathcal{O}(1)$ time by applying the t- and f-tests to determine whether a concept drift occurred. To assess OPTWIN, we compared it with 5 popular drift detectors in

11 different experiments. As a result, OPTWIN achieved higher F1-scores in most of our experiments. In fact, OPTWIN had the best F1-score (with statistical significance) while maintaining a similar drift-detection delay compared to the drift detectors suited for both classification and regression problems. Moreover, OPTWIN could speed up the overall OL pipeline of a CNN by 21% (compared to ADWIN) due to its low FP rate. To conclude, we conjecture that OPTWIN's characteristics can enable even higher speed-ups in scenarios where the drift identification triggers the re-training of complex models.

4.8 Supplementary Material

4.8.1 Proof

4.8.1.1 Definition of concepts

- δ: confidence level defined by the user, $\delta \in (0, 1)$.
- W: sliding window of error rates.
- w_{proof}: minimum size of W to identify drifts of ρ magnitude.
- w_{min}: minimum W size, with $w_{min} = 30$.
- w_{max}: maximum W size, with $w_{max} \in [w_{min}, \infty)$.
- μ_W: mean of W.
- σ_W: standard deviation of W.
- ν: optimal splitting percentage of W, with $\nu \in (0, 1)$.
- ν_{split}: optimal splitting point of W, with $\nu_{split} = \lfloor \nu\,|W| \rfloor$
- W_{hist}: historical errors from W, defined as $W_{0:\nu_{split}}$.
- W_{new}: new errors from W, defined as $W_{\nu_{split}:|W|-1}$.
- ρ: robustness parameter, i.e., the ratio that $\mu_{W_{new}}$ has to vary in relation to $\sigma_{W_{hist}}$ to count as a concept drift, with $\rho\ in(0, \infty)$.
- ρ_{temp}: temporary ρ, defined by solving Equation 4.1, with $\nu = 0.5$.
- $X = r_1, r_2, ..., r_i, ...$: data-stream of real numbers r_i.

4.8.1.2 Proof of Theorem 4.5.1

First, let us recall Theorem 4.5.1.

Theorem 4.5.1.

- ***False Positive Bound.*** *At every step, if μ_W and σ^2_W remain constant within W, OPTWIN will flag a concept drift at this step with a confidence of at most 1-δ.*
- ***False Negative Bound*** *(for mean drift with large enough W). For any partitioning of W into two sub-windows $W_{hist}\,W_{new}$, with $|W| \geq w_{proof}$ and W_{new} containing the most recent elements, if $\mu_{hist} - \mu_{new} > \rho\,\sigma_{hist}$, then, with confidence δ, OPTWIN flags a concept drift in at most $|W| - \nu_{split}$ steps.*

- ***False Negative Bound*** *(for mean drift with small W). For any partitioning of W into two sub-windows W_{hist} W_{new}, with $w_{min} \leq |W| < w_{proof}$ and W_{new} containing the most recent elements, if $\mu_{hist} - \mu_{new} > \rho_{temp}\,\sigma_{hist}$, then, with confidence δ, OPTWIN flags a concept drift in at most $\frac{|W|}{2}$ steps.*
- ***False Negative Bound*** *(for standard-deviation drift with any W). For any partitioning of W into two sub-windows W_{hist} W_{new}, with $|W| \geq w_{min}$ and W_{new} containing the most recent elements, if $\frac{\sigma^2_{new}}{\sigma^2_{hist}} > f_ppf(\delta', \nu\,|W| - 1, (1-\nu)\,|W| - 1)$, then, with confidence δ, OPTWIN flags a concept drift in at most ν_{split} steps.*

As seen above, Theorem 4.5.1 is based on ρ, which we defined in Equation 4.1 and we recall it below.

$$\rho = t_ppf(\delta', df)\sqrt{\frac{1}{\nu|W|} + \frac{f_ppf(\delta', \nu\,|W|-1, (1-\nu)\,|W|-1)}{(1-\nu)\,|W|}} \tag{4.1}$$

Proof.

- **Part 1:** In each iteration of OPTWIN, we apply the t- and the f-tests to identify the concept drifts. Therefore, the false-positive bounds of OPTWIN are directly drawn from those tests. Thus, with $\delta' = \delta^{\frac{1}{4}}$ as the confidence for each of the four tests, we identify a false-positive drift with a confidence of at most $1 - \delta$.

- **Part 2:** Considering an unequal-variance t-test (Ruxton, 2006) by comparing W_{hist} and W_{new}, we have:

$$t_value = \frac{\mu_{hist} - \mu_{new}}{\sqrt{\frac{\sigma^2_{hist}}{|W_{hist}|} + \frac{\sigma^2_{new}}{|W_{new}|}}} \tag{4.2}$$

The nominator on the right represents the difference observed between the means of W_{hist} and W_{new}. If this difference is large enough, we can statistically guarantee that both sets have distributions with different means. To simplify the equation, we represent this difference in terms of σ_{hist}. Therefore, we set $\mu_{hist} - \mu_{new} = \rho\,\sigma_{hist}$ with ρ being a user-defined robustness parameter. Thus, by applying this substitution and passing the denominator to the other side of the equation, we have:

$$\rho\,\sigma_{hist} = t_value\sqrt{\frac{\sigma^2_{hist}}{|W_{hist}|} + \frac{\sigma^2_{new}}{|W_{new}|}} \tag{4.3}$$

Considering ν as the splitting point of W into W_{hist} and W_{new}, we have $|W| = \nu\,|W_{hist}| + (1-\nu)\,|W_{new}|$, and therefore:

$$\rho\,\sigma_{hist} = t_value\sqrt{\frac{\sigma^2_{hist}}{\nu\,|W|} + \frac{\sigma^2_{new}}{(1-\nu)\,|W|}} \tag{4.4}$$

By our definition of a concept drift, σ_{new} and σ_{hist} must follow the same distribution (otherwise a drift occurred). Furthermore, we also consider that they follow the f-distribution, which is usually the case for standard deviations.

Thus, consider the definition of the test statistic for the f-test (Mahbobi and Tiemann, 2016) as shown below:

$$f_value = \frac{\sigma^2_{new}}{\sigma^2_{hist}} \tag{4.5}$$

We can now determine σ_{new}'s upper bound in terms of σ_{hist} and by defining f_factor as the highest value that the f_value can have while maintaining the null hypothesis of the f-test with confidence δ'.

$$\sigma^2_{hist} f_factor \geq \sigma^2_{new} \tag{4.6}$$

In this case, the f_factor can be calculated using the f_ppf, the PPF of the F-curve, by taking a confidence of δ' and $|W_{new}| - 1$ and $|W_{hist}| - 1$ as the degrees of freedom for the numerator and the denominator, respectively (cf. Equation 4.5). Thus, we obtain:

$$f_factor = f_ppf(\delta', \nu\,|W| - 1, (1-\nu)\,|W| - 1) \tag{4.7}$$

Therefore, to later simplify Equation 4.4, we substitute σ_{new} with its upper bound as calculated by the f-test on Equation 4.6:

$$\rho\,\sigma_{hist} = t_value \sqrt{\frac{\sigma^2_{hist}}{\nu\,|W|} + \frac{\sigma^2_{hist} f_factor}{(1-\nu)\,|W|}} \tag{4.8}$$

Next, we can simplify the equation by removing σ_{hist} from both sides as follows:

$$\rho = t_value \sqrt{\frac{1}{\nu\,|W|} + \frac{f_factor}{(1-\nu)\,|W|}} \tag{4.9}$$

Then, we calculate the t_value in Equation 4.10 analogously by using the PPF of the T-curve.

$$t_value = t_ppf(\delta', df) \tag{4.10}$$

We consider a confidence level of δ' and the degrees of freedom df for the unequal-variance t-test (cf. Equation 4.11).

$$df = \frac{\left(\frac{\sigma^2_{hist}}{|W_{hist}|} + \frac{\sigma^2_{new}}{|W_{new}|}\right)^2}{\frac{\left(\frac{\sigma^2_{new}}{|W_{hist}|}\right)^2}{|W_{hist}|-1} + \frac{\left(\frac{\sigma^2_{new}}{|W_{new}|}\right)^2}{(|W_{new}|)-1}} \tag{4.11}$$

By considering Equation 4.11 in our setting with $|W_{hist}| = \nu\,|W|$ and $|W_{new}| = (1-\nu)\,|W|$ and our upper bound for $\sigma_{new} \leq \sigma_{hist} f_factor$ (cf. Equation 4.6), we reach Equation 4.12, which depicts our calculation of the degrees of freedom df.

$$df = \frac{\left(\frac{1}{\nu|W|} + \frac{f_ppf}{(1-\nu)|W|}\right)^2}{\frac{\left(\frac{1}{\nu|W|}\right)^2}{(\nu|W|)-1} + \frac{\left(\frac{f_ppf}{(1-\nu)|W|}\right)^2}{((1-\nu)|W|)-1}} \tag{4.12}$$

Finally, by replacing t_value on Equation 4.9 by the one calculated on Equation 4.10, we reach Equation 4.1 introduced in Section 4.5.2. □

We remark that Equation 4.1 only depends on δ', ρ, $|W|$ and ν. Recall that δ' is $\delta^{\frac{1}{4}}$ which is provided by the user; ρ is also given by the user; and $|W|$ increases linearly until w_{max}. We can then calculate the highest value ν based on each $|W|$ that solves Equation 4.1.

Therefore, considering the split $W = W_{hist}\, W_{new}$ with $W_{hist} = W_{0:\nu\,|W|}$ and $W_{new} = W_{\nu\,|W|:|W|-1}$, by applying the t-test, we guarantee that if μ_{hist} and μ_{new} diverge more than $\rho\,\sigma_{hist}$, we identify this divergence with a confidence of δ. Moreover, considering our OL setting, it takes no more than $(1-\nu)|W|$ iterations for W to be divided into $W_{hist}\, W_{new}$ after the drift occurred.

- **Part 3:** By setting $\nu = 0.5$ in Equation 4.9, we have:

$$\rho_{temp} = t_value\sqrt{\frac{1}{0.5\,|W|} + \frac{f_factor}{0.5\,|W|}} \tag{4.13}$$

  With $\nu = 0.5$ and ρ_{temp} as calculated above, we can guarantee that if μ_{hist} and μ_{new} diverge more than $\rho_{temp}\,\sigma_{hist}$, we identify the divergence with a confidence of δ (as guaranteed by the test procedure). Similarly to Part 2 of the proof, we achieve this in at most $(1-\nu)\,|W|$ iterations, in this case, $\frac{|W|}{2}$ iterations. □

- **Part 4:** This bound is directly obtained from the f-test. By simply applying the f-test to compare two sub-windows of W, we guarantee the given confidence value, in this case δ. Again, it takes no more than $(1-\nu)\,|W|$ iterations for W to be divided into $W_{hist}\, W_{new}$ after the drift occurred. □

4.8.2 Additional Plots

This section presents the remaining plots related to Tables 4.1, 4.2, and 4.3.

In Figure 4.5, we cannot observe the same effect of ρ in OPTWIN's delay detection that was observed in Figure 4.2. Nevertheless, we can still observe the high FP rate of EDDM, ECDD , and ADWIN compared to the other drift detectors. We note that STEPD's high FP rate comes from a few runs with dozens of FPs.

Figures 4.6 and 4.7 illustrate, respectively, the sudden and gradual drift detection experiments using non-binary data. In these experiments, we can clearly observe the excellent results obtained by OPTWIN in terms of drifts detected and the delay of those detections, independently of ρ. When analyzing ADWIN, we can see in Figure 4.6 how it correctly identifies the concept drifts but ends up producing FPs while its sliding window is not fully adapted to the new distribution. On the other hand, STEPD produced almost random guesses throughout both experiments, with multiple FPs.

Next, Figures 4.8 and 4.9 illustrate the results over the STAGGER and the Random RBF datasets, respectively. In Figure 4.8 we see that, besides STEPD, all drift detectors could successfully identify the concept drifts with few (or none) FPs. Differently, the Random RBF dataset was more challenging to the drift detectors. Consequently, in Figure 4.9 we can visualize that STEPD and ECDD once more produced almost random guesses on the location of the concept drifts, with multiple FPs. Besides those detectors, none of the others could successfully identify the third concept drift, which we assume to be due to an extremely subtle change in the data distribution. At last, we highlight that in the Random RBF experiment, the accuracy of the learner sometimes decreased even after the correct identification of the concept drift due to the new distribution being more challenging for the NB classifier to represent.

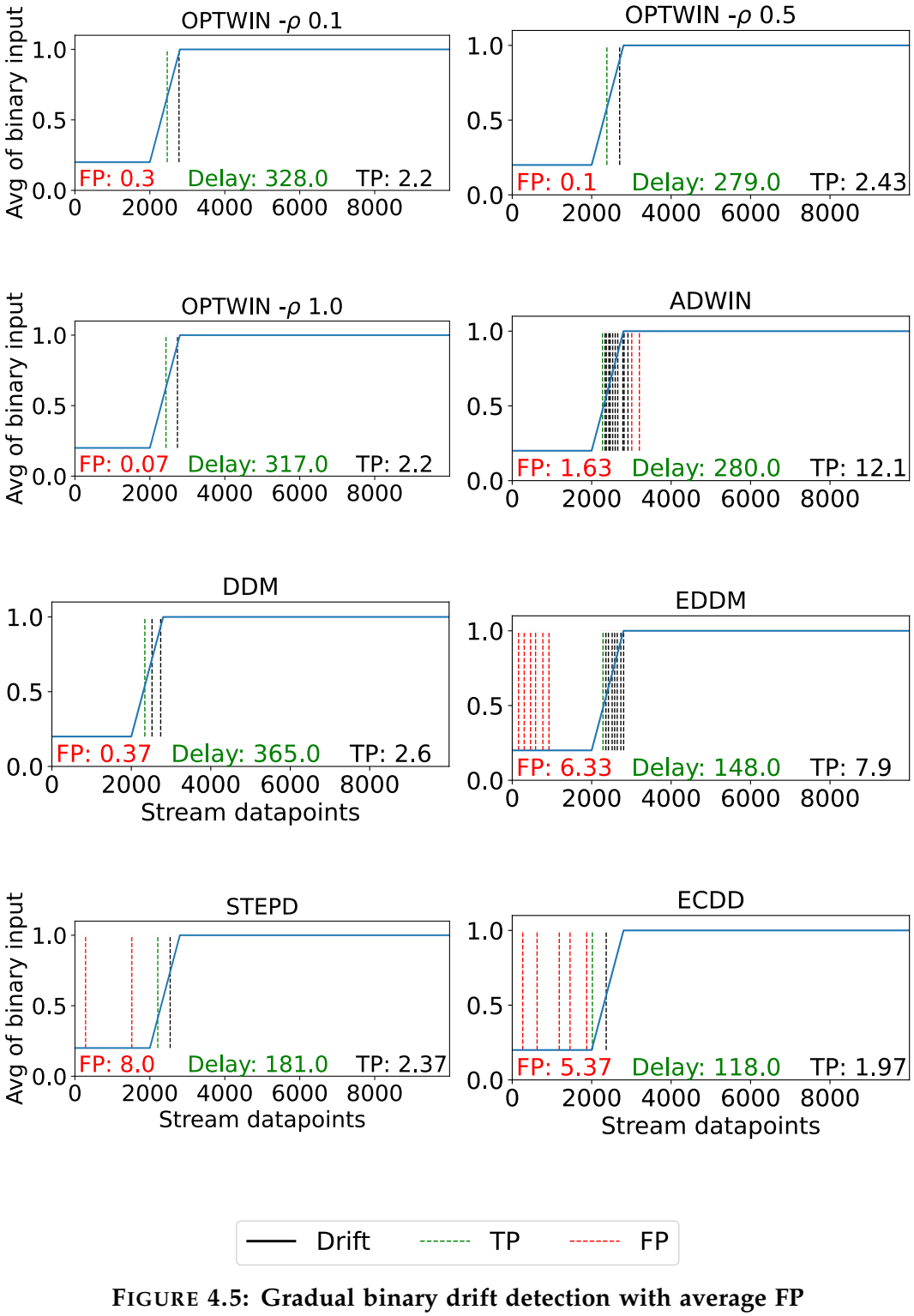


FIGURE 4.5: Gradual binary drift detection with average FP rates compared to drift-detection delays.

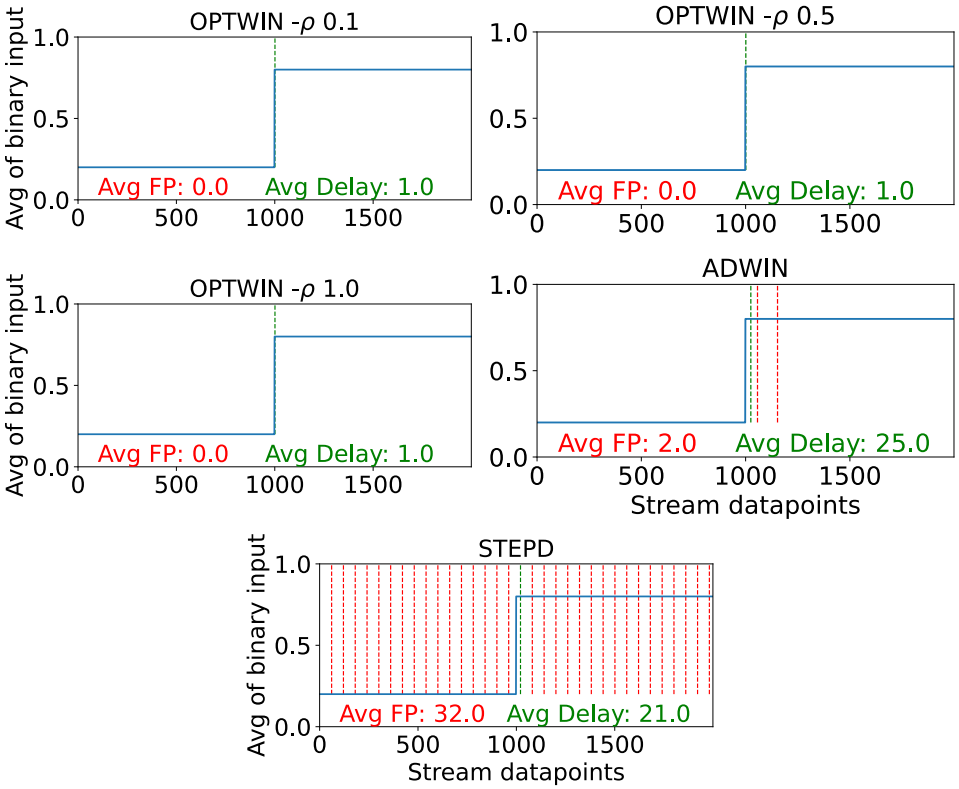


FIGURE 4.6: Sudden non-binary drift detection with average FP rates compared to drift-detection delays.

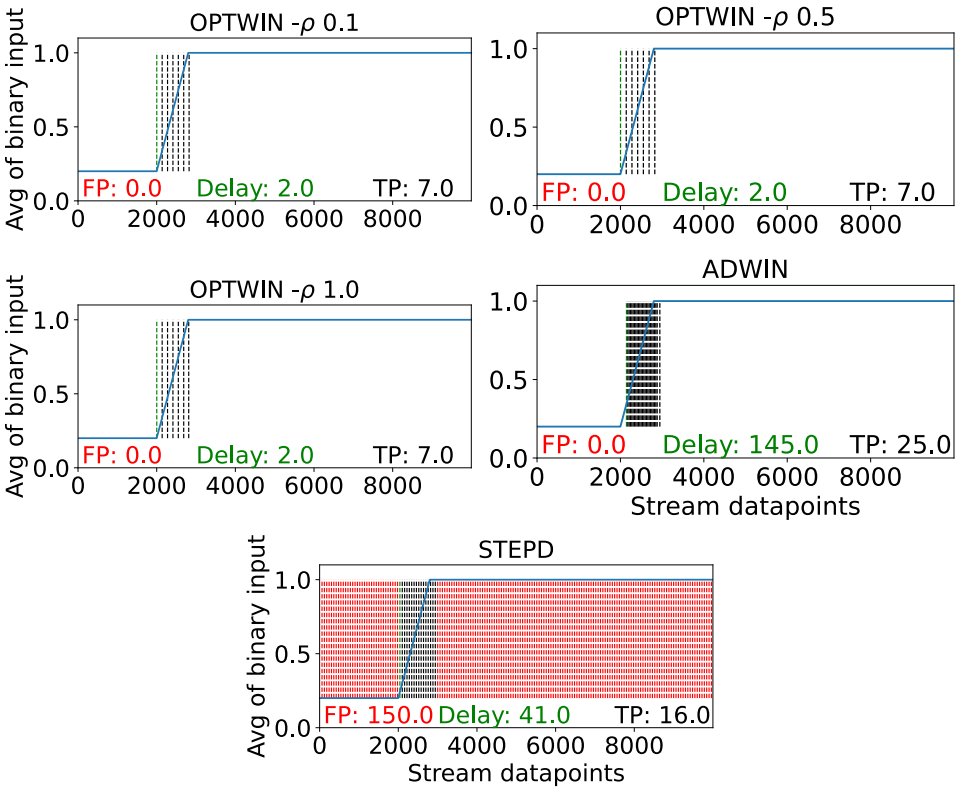


FIGURE 4.7: Gradual non-binary drift detection with average FP rates compared to drift-detection delays.

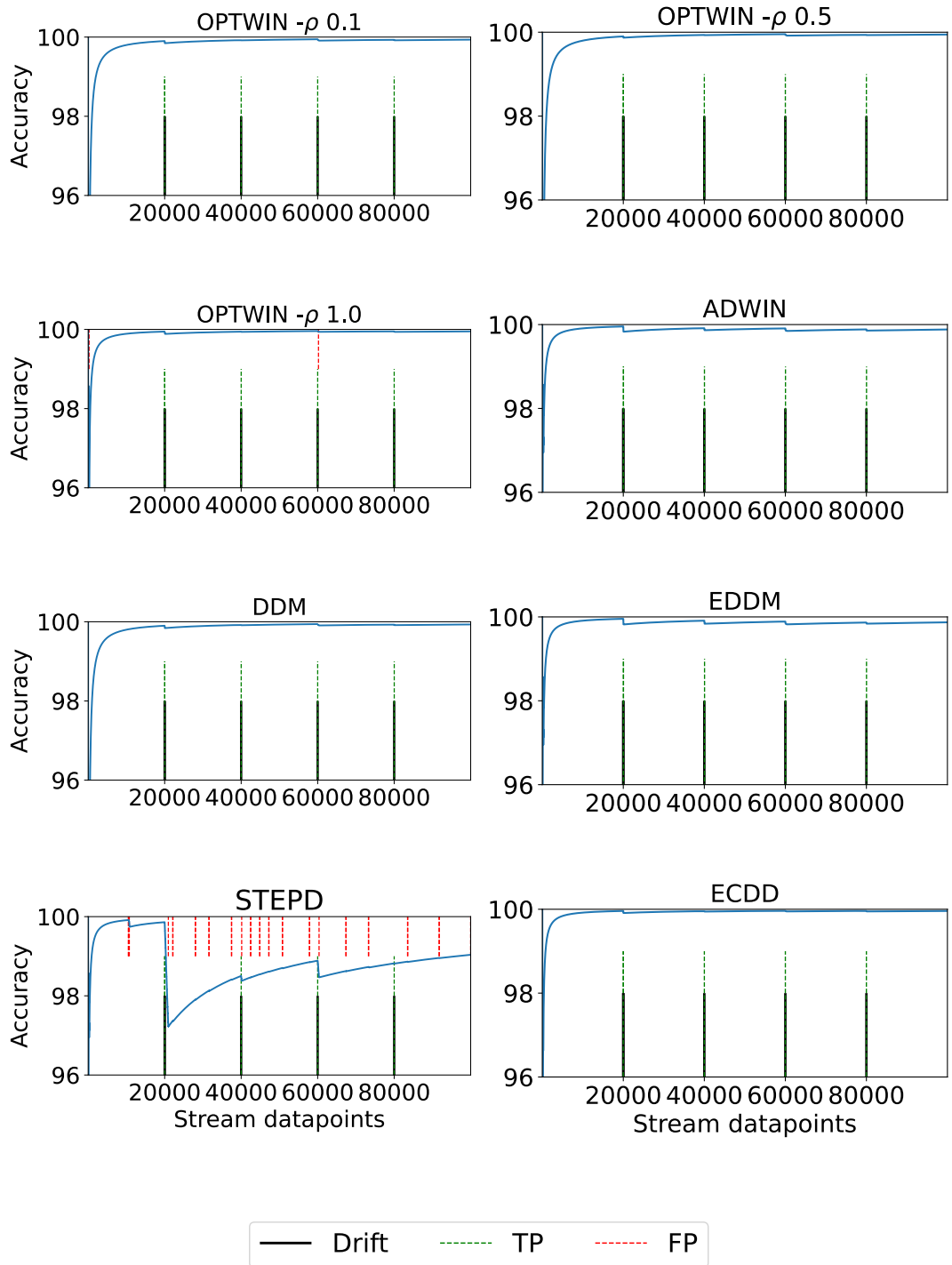


FIGURE 4.8: TP and FP rates for drift detection on the STAGGER dataset with sudden concept drifts.

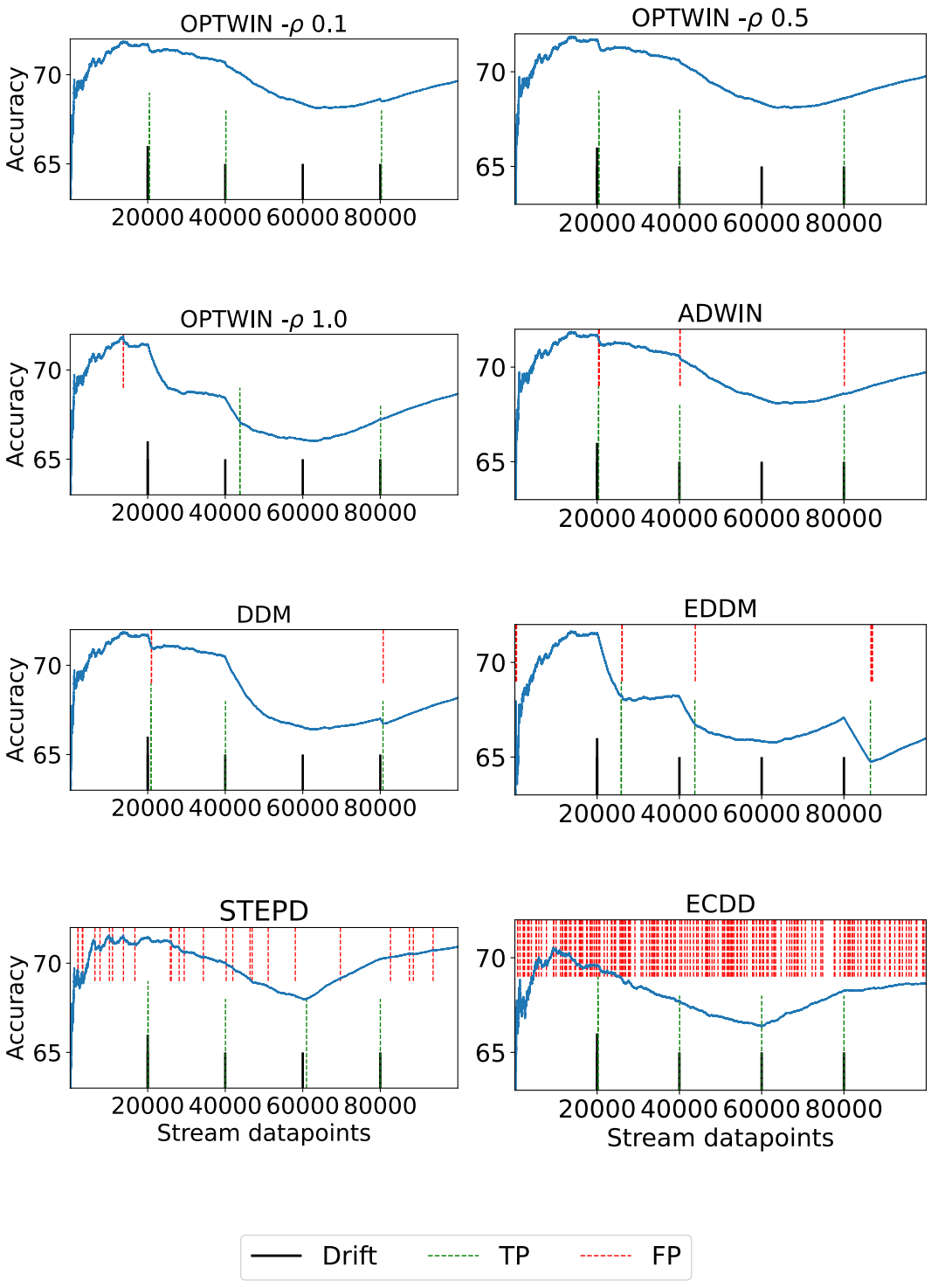


FIGURE 4.9: TP and FP rates for drift detection on the RandomRBF dataset with sudden concept drifts.